

```

# =====
# Student Performance Prediction using Random Forest
# Project: Enhancing Student Performance Prediction using Random Forest
#           A Comparative Study with Regularization-based Linear Regression
# Dataset: 300 students | Features: Kehadiran, Part, Tugas, UTS, UAS
# Target : NilaiAkhir (Final Grade)
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.model_selection import train_test_split, cross_val_score, GridSearch
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler

# =====
# MODULE 1: DATA LOADING & PREPROCESSING
# =====

print("=" * 65)
print("  MODULE 1: DATA LOADING & PREPROCESSING")
print("=" * 65)

# — Load dataset —————
df = pd.read_excel("student_dataset_300.xlsx", sheet_name="Student Data")

# Rename columns to short names for convenience
df.columns = ["Student_ID", "Kehadiran", "Part", "Tugas", "UTS", "UAS",
              "NilaiAkhir", "Grade"]

print(f"\nDataset shape : {df.shape}")
print(f"Total students: {len(df)}")
print("\nFirst 5 rows:")
print(df.head())
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())

```

```

# — Features & Target —————
FEATURES = ["Kehadiran", "Part", "Tugas", "UTS", "UAS"]
TARGET    = "NilaiAkhir"

X = df[FEATURES]
y = df[TARGET]

print(f"\nFeatures : {FEATURES}")
print(f"Target      : {TARGET}")

# =====
# MODULE 2: EXPLORATORY DATA ANALYSIS & FEATURE CORRELATION
# =====

print("\n" + "=" * 65)
print("  MODULE 2: EXPLORATORY DATA ANALYSIS & FEATURE CORRELATION")
print("=" * 65)

print("\nDescriptive Statistics:")
print(df[FEATURES + [TARGET]].describe().round(2))

# — Correlation matrix —————
corr = df[FEATURES + [TARGET]].corr().round(4)
print("\nCorrelation Matrix:")
print(corr)

print(f"\nHighest correlation with NilaiAkhir: "
      f"{corr[TARGET].drop(TARGET).idxmax()} "
      f"({corr[TARGET].drop(TARGET).max():.4f})")

# — Grade distribution —————
print("\nGrade Distribution:")
print(df["Grade"].value_counts().sort_index())

# — Plots —————
fig, axes = plt.subplots(2, 3, figsize=(15, 9))
fig.suptitle("Feature Distributions", fontsize=14, fontweight="bold")
for ax, feat in zip(axes.flatten(), FEATURES + [TARGET]):
    ax.hist(df[feat], bins=20, color="#2E75B6", edgecolor="white", alpha=0.85)
    ax.set_title(feat)
    ax.set_xlabel("Score")
    ax.set_ylabel("Count")
plt.tight_layout()
plt.savefig("feature_distributions.png", dpi=150)
plt.close()
print("\n[Saved] feature_distributions.png")

```

```

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="Blues",
            linewidths=0.5, square=True)
plt.title("Feature Correlation Heatmap", fontweight="bold")
plt.tight_layout()
plt.savefig("correlation_heatmap.png", dpi=150)
plt.close()
print("[Saved] correlation_heatmap.png")

# =====
# MODULE 3: REGULARIZATION-BASED LINEAR REGRESSION (Baseline)
# =====

print("\n" + "=" * 65)
print("  MODULE 3: REGULARIZATION-BASED LINEAR REGRESSION (Baseline)")
print("=" * 65)

TRAIN_SIZES = [0.10, 0.20, 0.30, 0.40, 0.50, 0.60]
ALPHAS       = [0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1, 5]
N_RUNS       = 10    # runs per configuration for stable averages

scaler = StandardScaler()

def evaluate(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return mse, rmse, mae, r2

# Store best results per model
baseline_results = {}

for model_name, ModelClass in [("Ridge", Ridge),
                               ("Lasso", Lasso),
                               ("ElasticNet", ElasticNet)]:
    best = {"mse": np.inf}
    avg_metrics_per_alpha = []

    for alpha in ALPHAS:
        run_metrics = []
        for ts in TRAIN_SIZES:
            for _ in range(N_RUNS):
                X_tr, X_te, y_tr, y_te = train_test_split(
                    X, y, train_size=ts, random_state=None)
                Xs_tr = scaler.fit_transform(X_tr)

```

```

        Xs_te = scaler.transform(X_te)
        m = ModelClass(alpha=alpha)
        m.fit(Xs_tr, y_tr)
        mse, rmse, mae, r2 = evaluate(y_te, m.predict(Xs_te))
        run_metrics.append([mse, rmse, mae, r2])
        if mse < best["mse"]:
            best = {"mse": mse, "rmse": rmse, "mae": mae,
                    "r2": r2, "alpha": alpha, "ts": ts}

    avg = np.mean(run_metrics, axis=0)
    avg_metrics_per_alpha.append(avg)

baseline_results[model_name] = {
    "best": best,
    "avg_per_alpha": np.array(avg_metrics_per_alpha)
}

print(f"\n{model_name} - Best result:"
      f"  alpha={best['alpha']}, train_size={int(best['ts']*100)}%"
      f"  MSE={best['mse']:.4f}  RMSE={best['rmse']:.4f}"
      f"  MAE={best['mae']:.4f}  R2={best['r2']:.4f}")

# =====
# MODULE 4: RANDOM FOREST MODEL CONSTRUCTION & TUNING
# =====

print("\n" + "=" * 65)
print("  MODULE 4: RANDOM FOREST MODEL CONSTRUCTION & TUNING")
print("=" * 65)

# — 4a. Hyperparameter Grid Search —————
print("\nRunning GridSearchCV for Random Forest hyperparameter tuning...")

param_grid = {
    "n_estimators" : [50, 100, 200],
    "max_depth"     : [None, 5, 10, 20],
    "min_samples_split": [2, 5, 10],
    "max_features"  : ["sqrt", "log2"]
}

rf_base = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(
    rf_base, param_grid,
    scoring="neg_mean_squared_error",
    cv=5, n_jobs=-1, verbose=0
)

X_scaled = scaler.fit_transform(X)
grid_search.fit(X_scaled, y)

```

```

best_params = grid_search.best_params_
print(f"\nBest hyperparameters: {best_params}")
print(f"Best CV MSE           : {-grid_search.best_score_:.4f}")

# — 4b. Train final RF model with best params —————
rf_model = RandomForestRegressor(**best_params, random_state=42)

# — 4c. Evaluate RF across same training sizes as baselines —————
print("\nEvaluating Random Forest across training sizes...")

rf_results_by_size = []
for ts in TRAIN_SIZES:
    run_metrics = []
    for _ in range(N_RUNS):
        X_tr, X_te, y_tr, y_te = train_test_split(
            X, y, train_size=ts, random_state=None)
        Xs_tr = scaler.fit_transform(X_tr)
        Xs_te = scaler.transform(X_te)
        rf_model.fit(Xs_tr, y_tr)
        mse, rmse, mae, r2 = evaluate(y_te, rf_model.predict(Xs_te))
        run_metrics.append([mse, rmse, mae, r2])
    avg = np.mean(run_metrics, axis=0)
    rf_results_by_size.append([ts] + list(avg))
    print(f"  Train {int(ts*100):2d}% → "
          f"MSE={avg[0]:.4f}  RMSE={avg[1]:.4f}  "
          f"MAE={avg[2]:.4f}  R²={avg[3]:.4f}")

rf_df = pd.DataFrame(rf_results_by_size,
                     columns=["TrainSize", "MSE", "RMSE", "MAE", "R2"])

# =====
# MODULE 5: MODEL EVALUATION & COMPARATIVE ANALYSIS
# =====

print("\n" + "=" * 65)
print("  MODULE 5: MODEL EVALUATION & COMPARATIVE ANALYSIS")
print("=" * 65)

# — Final head-to-head on 60/40 split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42)
X_tr_sc = scaler.fit_transform(X_train)
X_te_sc = scaler.transform(X_test)

models = {
    "Ridge"           : Ridge(alpha=best_params.get("alpha", 0.001)),

```

```

    "Lasso"          : Lasso(alpha=0.001),
    "ElasticNet"     : ElasticNet(alpha=0.001),
    "RandomForest": RandomForestRegressor(**best_params, random_state=42)
}

print("\n— Final Evaluation (60% train / 40% test) —")
print(f"{'Model':<15} {'MSE':>8} {'RMSE':>8} {'MAE':>8} {'R²':>8}")
print("-" * 50)

final_results = {}
for name, model in models.items():
    model.fit(X_tr_sc, y_train)
    preds = model.predict(X_te_sc)
    mse, rmse, mae, r2 = evaluate(y_test, preds)
    final_results[name] = {"MSE": mse, "RMSE": rmse, "MAE": mae, "R2": r2}
    print(f"{name:<15} {mse:>8.4f} {rmse:>8.4f} {mae:>8.4f} {r2:>8.4f}")

# — 5-fold Cross-Validation —————
print("\n— 5-Fold Cross-Validation R² Scores —")
for name, model in models.items():
    scores = cross_val_score(model, X_scaled, y, cv=5, scoring="r2")
    print(f"{name:<15} mean={scores.mean():.4f} std={scores.std():.4f}")

# — Comparison bar chart —————
metrics_df = pd.DataFrame(final_results).T
fig, axes = plt.subplots(1, 4, figsize=(16, 5))
fig.suptitle("Model Comparison (60% Train / 40% Test)", fontsize=13, fontweight="bold")
colors = ["#5B9BD5", "#ED7D31", "#A9D18E", "#FF0000"]

for ax, metric in zip(axes, ["MSE", "RMSE", "MAE", "R2"]):
    bars = ax.bar(metrics_df.index, metrics_df[metric], color=colors, edgecolor="black")
    ax.set_title(metric, fontweight="bold")
    ax.set_xticklabels(metrics_df.index, rotation=20, ha="right", fontsize=9)
    for bar in bars:
        ax.text(bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.002,
                f"{bar.get_height():.3f}",
                ha="center", va="bottom", fontsize=8)

plt.tight_layout()
plt.savefig("model_comparison.png", dpi=150)
plt.close()
print("\n[Saved] model_comparison.png")

# — Actual vs Predicted —————
rf_model.fit(X_tr_sc, y_train)
rf_preds = rf_model.predict(X_te_sc)

```

```

plt.figure(figsize=(6, 6))
plt.scatter(y_test, rf_preds, alpha=0.6, color="#2E75B6", edgecolors="white", s=5
mn, mx = y_test.min(), y_test.max())
plt.plot([mn, mx], [mn, mx], "r--", linewidth=1.5, label="Perfect Prediction")
plt.xlabel("Actual NilaiAkhir")
plt.ylabel("Predicted NilaiAkhir")
plt.title("Random Forest: Actual vs Predicted", fontweight="bold")
plt.legend()
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=150)
plt.close()
print("[Saved] actual_vs_predicted.png")

# =====
# MODULE 6: FEATURE IMPORTANCE ANALYSIS & RESULT REPORTING
# =====

print("\n" + "=" * 65)
print("  MODULE 6: FEATURE IMPORTANCE ANALYSIS & RESULT REPORTING")
print("=" * 65)

rf_model.fit(X_tr_sc, y_train)
importances = pd.Series(rf_model.feature_importances_, index=FEATURES)
importances = importances.sort_values(ascending=False)

print("\nFeature Importances (Mean Decrease in Impurity):")
for feat, imp in importances.items():
    bar = "█" * int(imp * 50)
    print(f"  {feat:<12} {imp:.4f}  {bar}")

plt.figure(figsize=(7, 4))
colors_imp = ["#1E4D8C" if i == 0 else "#2E75B6" if i == 1 else "#5B9BD5"
               for i in range(len(importances))]
bars = plt.barh(importances.index[::-1], importances.values[::-1],
                color=colors_imp[::-1], edgecolor="white")
for bar in bars:
    plt.text(bar.get_width() + 0.002, bar.get_y() + bar.get_height() / 2,
             f"{bar.get_width():.4f}", va="center", fontsize=9)
plt.xlabel("Importance Score")
plt.title("Random Forest – Feature Importance", fontweight="bold")
plt.tight_layout()
plt.savefig("feature_importance.png", dpi=150)
plt.close()
print("\n[Saved] feature_importance.png")

# — Final Summary Report —————

```

```

print("\n" + "=" * 65)
print("  FINAL SUMMARY REPORT")
print("=" * 65)
print(f"\n  Dataset          : 300 students, 5 features → NilaiAkhir")
print(f"  Best RF Params : {best_params}")
print(f"\n  {'Model':<15}  {'MSE':>7}  {'RMSE':>7}  {'MAE':>7}  {'R²':>7}")
print("    " + "-" * 48)
for name, res in final_results.items():
    marker = "  ◀ BEST" if name == "RandomForest" else ""
    print(f"    {name:<15}  {res['MSE']:>7.4f}  {res['RMSE']:>7.4f}"
          f"    {res['MAE']:>7.4f}  {res['R2']:>7.4f}{marker}")

best_model = max(final_results, key=lambda k: final_results[k]["R2"])
print(f"\n  Best performing model: {best_model}")
print(f"  Most important feature: {importances.index[0]} ({importances.iloc[0]:.4}")
print(f"\n  Output files saved:")
for f in ["feature_distributions.png", "correlation_heatmap.png",
         "model_comparison.png", "actual_vs_predicted.png",
         "feature_importance.png"]:
    print(f"      • {f}")
print("=" * 65)

```